
Representation Drift Compensation: A Near-Zero Inference Cost Enhancement for LLM Decomposition

Xinhao Huang¹ You-Liang Huang^{1 2} Zeyi Wen^{1 3}

Abstract

While low-rank decomposition offers potential for reducing LLM parameters, maintaining the original capabilities remains a significant challenge. In this work, we identify and formalize a key overlooked issue in LLM decomposition: *representation drift*. We show that approximation errors introduced by decomposition propagate and amplify non-linearly through the deep layers of the transformer architecture, progressively distorting internal representations and degrading downstream performance. To mitigate this, we introduce a conceptually simple but principled compensation mechanism, named “Decomper”, that operates by suppressing error at its source. By learning to align the output distribution of decomposed transformer blocks with their original counterparts, our method effectively counteracts representation drift, achieving notable performance recovery with near-zero inference overhead. Extensive experiments in OPT, LLaMA-2/3, and Qwen exhibit remarkable improvements. For instance, on LLaMA-3-8B and OPT-13B at 40% compression, perplexity is reduced by more than 70% while reasoning task accuracy improves by over 10%. Our code is available at this [URL](#).

1. Introduction

The scaling of large language models (LLMs) has delivered impressive capabilities but also created formidable challenges for their practical deployment due to substantial computational and memory demands (Kaplan et al., 2020; Miao et al., 2023). Consequently, model compression has become an essential research direction, with techniques

like pruning (Frantar & Alistarh, 2023; Sun et al., 2024; Ashkboos et al., 2024; An et al., 2024; Ma et al., 2023), quantization (Frantar et al., 2023; Lin et al., 2024; Dettmers et al., 2023), and low-rank decomposition (Hsu et al., 2022; Yuan et al., 2023; Wang et al., 2025c;b; Yu & Wu, 2023; Chavan et al., 2024) being actively explored to reduce model size and accelerate inference.

Among these techniques, low-rank decomposition offers an approach to parameter reduction by approximating weight matrices with the product of two smaller matrices, typically achieved through Singular Value Decomposition (SVD). Despite its theoretical appeal for parameter reduction, low-rank decomposition often leads to notable performance degradation, which has hindered its practical adoption. Advanced methods primarily focus on minimizing the reconstruction error of individual weight matrices or their activations (Yu & Wu, 2023; Wang et al., 2025c; Huang et al., 2025). However, as our analysis reveals, such optimization targeting individual matrices is insufficient to prevent the accumulation and amplification of approximation errors throughout the network. We identify representation drift as the fundamental cause: minor approximation errors per layer are non-linearly amplified and accumulate across the deep network, leading to a substantial divergence in the latent representations from their original distribution. This drift manifests as a severe value reduction in the distribution of hidden states, ultimately degrading the model’s quality.

In this work, we first provide a diagnosis of this problem, including a theoretical analysis of the reconstruction loss and its propagation through transformer blocks. We then introduce Decomper, a simple and effective compensation mechanism designed to mitigate this drift directly. Our key insight is that learning a compensation term for each decomposed linear layer can effectively mitigate the approximation error, dramatically reducing the initial error injected into the computational graph. This mechanism is optimized to minimize block-level output divergence using a small calibration set (*e.g.*, 512 sequences with 256 tokens), is straightforward to implement, and adds no inference overhead as the learned compensation term can be fused into the linear operation.

Our comprehensive evaluations on the diverse model architectures (OPT, LLaMA-2, LLaMA-3, Qwen3 and Qwen2.5-

¹The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China ²Boston University, Boston, USA
³The Hong Kong University of Science and Technology, Hong Kong, China. Correspondence to: Zeyi Wen <wenzeyi@ust.hk>.

VL) across multiple scales (7B, 8B, 13B, and 30B) demonstrate that Decomper consistently outperforms existing methods in both structured pruning and low-rank decomposition. For instance, under the 40% compression ratio with OPT-13B, Decomper outperforms SVD-LLM, yielding a perplexity reduction from 68.0 to 15.88 and downstream task accuracy enhancement from 42.6% to 52.7%. Its efficacy extends to vision-language models, yielding remarkable improvements in COCO captioning metrics, notably a 23% CIDEr gain at 20% compression. In summary, our contributions are as follows:

- We identify and formalize the problem of representation drift as a key cause of performance degradation in low-rank decomposed LLMs.
- We propose a simple, effective, and near-zero-overhead compensation mechanism to mitigate this drift. To demonstrate its effectiveness, we quantitatively confirm the reduction in drift via Wasserstein distance.
- We empirically validate our method’s superiority over existing techniques across multiple models and benchmarks. We further demonstrate its inference speedups and synergistic value with quantization.

2. Related Work

This section reviews parameter reduction methods, specifically pruning and low-rank decomposition.

Network Pruning. Unstructured pruning exploits the inherent sparsity of model weights to eliminate specific connections, yet often struggles to achieve speedups due to hardware limitations (Frantar & Alistarh, 2023; Sun et al., 2024; Dong et al., 2024; Zhang & Pappan, 2025; Makni et al., 2025). Structured pruning removes entire channels or components for hardware acceleration (Ma et al., 2023; Ashkboos et al., 2024; van der Ouderaa et al., 2024; Lin et al., 2025; An et al., 2024). For instance, SliceGPT (Ashkboos et al., 2024) eliminates rows and columns via a transformation matrix, requiring extra adapters. FLAP (An et al., 2024) aligns layer outputs using a heuristic bias based on the mean contribution of pruned weights.

Low-Rank Decomposition. Unlike pruning, low-rank decomposition approximates dense weight matrices as the product of smaller low-rank matrices. Direct application of vanilla SVD often degrades performance significantly. To address this, FWSVD (Hsu et al., 2022) uses Fisher information to reconstruct weight matrices based on parameter importance. Recent truncation-aware methods link singular values to compression loss by considering activation reconstruction (Yuan et al., 2023; Huang et al., 2025; Wang et al., 2025c;b). Another line of work focuses on feature-space decomposition (Chavan et al., 2024), such as utilizing Atomic

Feature Mimicking (AFM) to decompose fully connected layer outputs (Yu & Wu, 2023; Ji et al., 2024). Most recently, Dobi-SVD (Wang et al., 2025b) minimizes reconstruction error via factor training, while Basis Sharing (Wang et al., 2025a) amortizes storage by sharing projection bases across adjacent layers. Despite these advancements, maintaining original capabilities after decomposition remains a significant challenge.

3. Representation Drift: Diagnosis and Mitigation

As illustrated in Figure 1, we propose a “compress-then-align” paradigm to mitigate representation drift. In this section, we present a theoretical analysis of representation drift and quantitatively demonstrate the effect of low-rank decomposition and compensation. Our analysis proceeds in three steps: (i) quantify per-layer decomposition residuals using SVD, (ii) propagate first-order perturbations through a stack of transformer blocks using first-order Taylor expansions and operator-norm bounds, and (iii) analyze the effect of compensation and its impact on expected error.

Notation. We summarize the key notations used throughout our analysis for clarity. $\mathcal{F}_l$: The mapping function of the l -th Transformer block. W, W_r : Original weight matrix and its low-rank approximation. R : Residual error matrix ($R = W - W_r$). X : Input activation for the linear transformation $Y = WX + b$. μ, Σ : Mean and covariance of X . H^l, H_r^l : Outputs of the original and decomposed l -th Transformer block. $\mathcal{D}_{\text{drift}}^l$: Representation drift at the l -th Transformer block. c : Learnable compensation vector.

3.1. Diagnosis: Error Propagation and Amplification

Linear layer reconstruction loss. Given a weight matrix $W \in \mathbb{R}^{m \times n}$, low-rank decomposition aims to reduce parameters while minimizing the reconstruction error. We approximate W by a rank- r matrix $W_r = AB$, where $A \in \mathbb{R}^{m \times r}$, $B \in \mathbb{R}^{r \times n}$ are factorized via Singular Value Decomposition (SVD) as $A = U_r \sqrt{\Sigma_r}$, $B = \sqrt{\Sigma_r} V_r^\top$. Here, U_r and V_r contain the top- r singular vectors, and $\sqrt{\Sigma_r}$ is a diagonal matrix of the square roots of the top- r singular values. This reduces the parameter count from $m \times n$ to $(m + n) \times r$.

Let W_r be its rank- r truncated approximation and $R = W - W_r$ the residual. The reconstruction loss for input activations X is defined with respect to the empirical distribution $\mathcal{D}$ of the calibration dataset. Treating X as a random variable drawn from $\mathcal{D}$, the reconstruction loss, governed by the mean μ and covariance Σ of X , is defined as:

$$\begin{aligned} \mathcal{L}(W_r) &= \mathbb{E}[\|WX - W_r X\|_2^2] = \mathbb{E}[\|RX\|_2^2] \\ &= \text{Tr}(R(\Sigma + \mu\mu^\top)R^\top) \\ &= \|R\mu\|_2^2 + \text{Tr}(R\Sigma R^\top). \end{aligned} \quad (1)$$

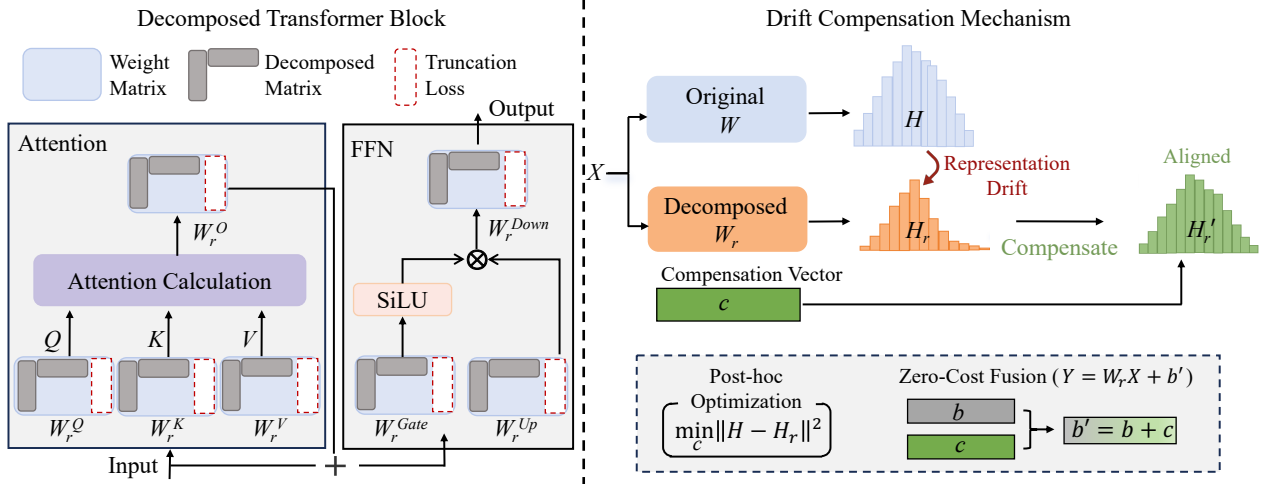


Figure 1. Decomper: A “compress-then-align” paradigm. We identify the latent representation drift that arises from low-rank decomposition techniques. Based on this finding, we propose a post-hoc compensation mechanism to mitigate the distribution drift.

This error provides a layer-wise baseline but crucially ignores how these errors interact and transform through the network’s non-linearities. The detailed derivation is provided in Appendix A.

Representation drift in the Transformer block. A Transformer block operates as a complex dynamical system, which consists of multiple linear projections in both self-attention module $SA(X) = W^O \cdot \text{Softmax}\left(\frac{W^Q X (W^K X)^T}{\sqrt{d_k}}\right) W^V X$ and feed-forward layers $FFN(X) = W^{down} (\sigma(W^{up} X) \odot W^{gate} X)$. After decomposition, each weight W^j becomes $W^j = W_r^j + R^j$. Although existing methods minimize the reconstruction error for each individual linear layer, the output of the entire decomposed Transformer block still exhibits significant deviation from that of the original block. This divergence stems not only from the accumulation of errors across multiple layers but also from the non-linear amplification of these errors through the deep network architecture.

Using LLaMA-3-8B as an example, we investigate the distribution of latent representations for the original model, the low-rank decomposition model, and the decomposed model enhanced with our compensation mechanism. We investigate the absolute values of the latent representations from Transformer blocks, displaying the 50th percentile, 95th percentile, and maximum value for each channel. For brevity, we illustrate the outputs from the 23rd Transformer block (out of 32). As shown in the first and second sub-figures of Figure 2, despite employing advanced low-rank decomposition techniques (e.g., truncation-aware data whitening minimizes linear layer reconstruction error), a significant drift of representations is observed between the decomposed and original models, particularly in the middle to tail blocks, which experience higher compression ratios. This drift re-

sults in inconsistent final outputs of the model, as the decomposed model yields latent representation values significantly lower than those of the original model. Such a pattern narrows the data distribution and negatively affects model quality. We provide additional visualizations for different model architectures in Appendix I.

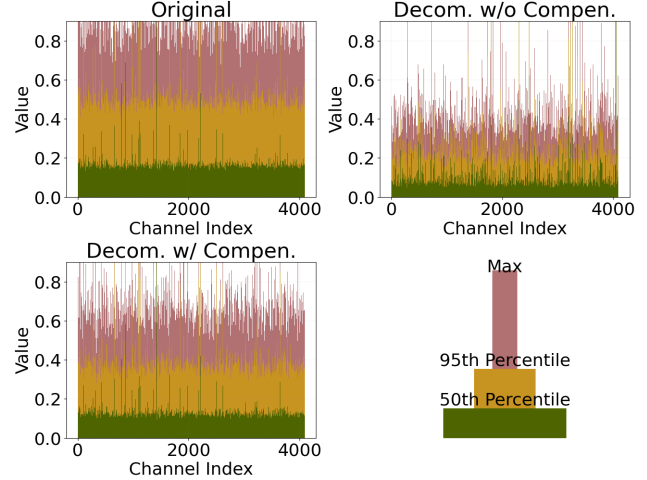


Figure 2. Latent representation drift in decomposition and our compensation on LLaMA-3-8B (23rd Block).

To analyze this phenomenon, we define *representation drift* as the divergence between the hidden state distributions of the original and decomposed models across transformer blocks. Let $H^l = \mathcal{F}(H^{l-1}; \{W^i\})$ be the output of the l -th transformer block in the original model. To capture the accumulation of errors, the decomposed model’s output at block l is defined recursively as $H_r^l = \mathcal{F}(H_r^{l-1}; \{W_r^i\})$, taking the *drifted* input from the previous decomposed block. $\mathcal{F}$ represents the composite function of self-attention, MLP,

residual connections, and LayerNorm operations. The original weights $W^i, i \in \{q, k, v, o, gate, up, down\}$ are replaced by their approximations W_r^i . The drift at block l is quantified by the expected divergence:

$$\mathcal{D}_{\text{drift}}^l = \mathbb{E}[\|H^l - H_r^l\|_2^2]. \quad (2)$$

Error propagation and amplification. To analyze the representation drift $\mathcal{D}_{\text{drift}}$, we employ a first-order Taylor expansion around the original weights W^i . The error propagates through the entire network. The divergence at the final block L can be expressed recursively:

$$\begin{aligned} \mathcal{D}_{\text{drift}}^L &= \mathbb{E}[\|H^L - H_r^L\|_2^2] \\ &\approx \mathbb{E}[\| \sum_{l=1}^L (\prod_{j=l+1}^L \underbrace{\frac{\partial H^j}{\partial H^{j-1}}}_{\text{Propagated Error}}) \cdot (\sum_{i \in \mathcal{W}} \underbrace{\frac{\partial \mathcal{F}^l}{\partial W^i} R^i}_{\text{Local Error}}) \|_2^2], \end{aligned} \quad (3)$$

where $R^i = W^i - W_r^i$ is the residual matrix for the i -th linear layer.

The *local error* captures the direct effect of decomposing the i -th linear layer in block l . For a linear layer $y = WX + b$, this error is RX , whose expected squared norm is $\mathbb{E}[\|RX\|_2^2] = \|R\mu\|_2^2 + \text{Tr}(R\Sigma R^\top)$, as derived in Equation 1. Equation 3 serves as a diagnostic tool, illustrating that the final error is a weighted sum of local errors where weights are products of Jacobians. Errors introduced in earlier layers (smaller l) undergo more non-linear amplification through the product of Jacobians $\prod_{j=l+1}^L \frac{\partial H^j}{\partial H^{j-1}}$, accumulating into a substantial propagated error that cannot be ignored. Empirically, as visualized in Figure 2, this propagated error significantly alters the representation distribution, manifesting as a drastic reduction in the magnitude of deep hidden states, making them more detrimental to the final output.

3.2. A Simple and Effective Compensation Mechanism

Output alignment via error suppression. As derivation in Section 3.1, our key idea is to compensate for the approximation error at each linear layer directly, preventing small local errors from accumulating across layers. We aim to find a correction term such that:

$$W_r X + \text{compensation} \approx WX. \quad (4)$$

A natural choice is to use a bias vector c , which is efficient to learn and add. Thus, for each decomposed layer, we learn a compensation vector c satisfying: $W_r X + c \approx WX$. This is equivalent to learning an approximation $c \approx (W - W_r)X = RX$. Therefore, introducing c effectively reduces the local error and mitigates the error propagation:

$$\mathbb{E}[\|WX - (W_r X + c)\|_2^2] \ll \mathbb{E}[\|RX\|_2^2] \quad (5)$$

By minimizing the local error at its source, our compensation mechanism *prevents the initial introduction of substantial error* into the computational graph. This drastically reduces the error available for propagation and amplification through subsequent layers, thereby mitigating the latent representation drift and aligning H^l more closely with H^l .

Proposition 3.1. *Given input X with mean μ and covariance Σ . For a decomposed linear layer with compensation vector c optimized to minimize the expected reconstruction error $\mathbb{E}[\|WX - (W_r X + c)\|_2^2]$, the expected error is upper-bounded by the variance component of the residual error.*

Proof. Let $R = W - W_r$. The objective is to find a constant vector c that minimizes $\mathbb{E}[\|RX - c\|_2^2]$. It is a standard statistical result that the optimal solution is the expected value: $c^* = \mathbb{E}[RX] = R\mu$. Substituting $c^* = R\mu$ back into the objective function, the residual error becomes $R(X - \mu)$. The expected squared norm is derived as: $\mathbb{E}[\|R(X - \mu)\|_2^2] = \text{Tr}(R \cdot \mathbb{E}[(X - \mu)(X - \mu)^\top] \cdot R^\top) = \text{Tr}(R\Sigma R^\top)$. This confirms that the variance component bounds the error. $\square$

Learned bias as a simple and practical solution. Although Proposition 3.1 provides a theoretical closed-form solution $c^* = R\mu$, its direct application in practice faces several challenges: (1) *The true input mean μ is unknown and must be estimated from a finite calibration set.* (2) *Non-linear functions within Transformer blocks cause significant distributional shift between layers, making a single, static estimate of μ insufficient for all inputs.*

A seemingly straightforward approach would be to compute this theoretical solution empirically by estimating $\hat{\mu} = \frac{1}{N} \sum x_i$ from the calibration data and setting $\hat{c} = R\hat{\mu}$. This mirrors the heuristic used in FLAP (An et al., 2024) for pruning, where the average contribution of pruned components is used as a bias correction. However, our analysis (Eq. 1) and practice reveal this heuristic is inadequate for low-rank decomposition. As established in Proposition 3.1, $c^* = R\mu$ only mitigates the mean error component $\|R\mu\|_2^2$, leaving the significant input-dependent variance term $\text{Tr}(R\Sigma R^\top)$ unaddressed. Furthermore, truncation-based low-rank decomposition introduces nonlinear operator errors that a static mean correction cannot fix. This explains why averaging tricks help pruning but are insufficient for decomposition.

Therefore, we formulate a block-level optimization problem using a small calibration dataset $\{x_k\}_{k=1}^N$ to empirically estimate the input distributions and account for non-linear interactions:

$$\hat{c} = \arg \min_c \frac{1}{N} \sum_{i=1}^N \|H_r^l(x_i; c) - H^l(x_i)\|_2^2 \quad (6)$$

where $H^l(x_k)$ is the target output from the original block given calibration sample x_k , and $H_r^l(x_k; c)$ is the output of the decomposed block. Note that H_r^l takes the drifting input from the previous decomposed layers, ensuring the optimization accounts for accumulated error. We solve this efficiently by optimizing c for each block via gradient descent, while keeping the original and decomposed model weights frozen, as detailed in Algorithm 1 (Appendix B).

Although the non-linearities in $\mathcal{F}$ render the optimization problem non-convex, freezing the massive pre-trained weights and optimizing only a single, low-dimensional bias vector drastically reduces the parameter search space from $\mathcal{O}(d^2)$ to $\mathcal{O}(d)$ (where d is the hidden dimension size). This massive reduction smooths the loss landscape, yielding a well-behaved optimization space. As empirically validated by the optimization trajectories detailed in Appendix Figure 5, the loss drops steeply within the first few batches and converges to a stable minimum, enabling the effective optimization of the compensation vector c . The resulting solution $\hat{c}$ provides a practical and highly effective means of minimizing the empirical output divergence. Consequently, while global optimality cannot be guaranteed, the procedure consistently converges to a strong local optimum that yields substantial performance recovery. Furthermore, this approach is highly lightweight, as it only requires optimizing low-dimensional vectors rather than full weight matrices.

Theoretically, our compensation mechanism not only reduces the immediate reconstruction error but also suppresses error propagation throughout the network. As visually demonstrated in Figure 2, the compensated representations align more closely with those of the original model. Specifically, the compensation effectively restores the range of representation values—which narrows considerably after low-rank decomposition—to distributions much closer to those of the original model. This realignment translates into improved prediction quality and greater reliability in the decomposed model. The improvement is especially notable in middle and later blocks with high compression ratios.

Quantifying representation alignment. To quantitatively verify the effectiveness of our drift compensation, we computed the Wasserstein distance between the latent distributions of the original and decomposed models. As shown in Table 1, compensation significantly pulls drifting representations back to the original manifold. On LLaMA-3-8B, compensation reduced the distance from 0.1324 to 0.0614 (a 53.6% reduction), and on LLaMA-2-7B from 0.2865 to 0.1749 (39.0% reduction).

Near-Zero-cost deployment. Following optimization, the learned compensation term, c , is seamlessly integrated into the linear layer by directly adding it to the existing bias term: $b' = b + c$. This design choice ensures several practical advantages: (1) the learned compensation terms are effi-

Table 1. Wasserstein distance comparison.

Model	w/o	w/	Improv.
LLaMA-3-8B	0.1324	0.0614	53.6%
LLaMA-2-7B	0.2865	0.1749	39.0%
LLaMA-2-13B	0.1820	0.0946	48.1%

ciently incorporated without requiring a separate parameter layer; (2) inference speed remains unaffected, as the vector addition operation is inherently fused into the existing linear computation and enjoys native hardware support; (3) the computational graph preserves its original structure, ensuring compatibility with existing deployment frameworks.

4. Experiments

This section presents a comparative analysis against strong baselines across diverse models and benchmarks (§ 4.1). We then ablate our core contribution, demonstrate its practical utility regarding inference speedup and quantization compatibility (§ 4.2), and discuss its distinctions from other parameter-update strategies (§ 4.3). Extended results, including higher compression ratios and generality to various techniques and vision-language models, are detailed in the Appendix due to space limitations.

Experimental settings. Models: We experiment with diverse models and scales, including LLaMA-2-7B/13B (Touvron et al., 2023), OPT-6.7B/13B/30B (Zhang et al., 2022), LLaMA-3-8B (Grattafiori et al., 2024), Qwen3-8B (Yang et al., 2025), and Qwen2.5-VL-7B-Instruct (Bai et al., 2025). We intentionally select both older (MHA-based) and newer (GQA-based) architectures to demonstrate generalizability. **Benchmarks:** We evaluate models on: WikiText2 perplexity (Merity et al., 2017); common-sense reasoning via LMEH (BoolQ, PIQA, WinoGrande, HellaSwag, ARC, OBQA) (Gao et al., 2021); knowledge-intensive MMLU (57 challenging tasks) (Hendrycks et al., 2020); and vision-language tasks on MS COCO (Lin et al., 2014). **Baselines:** We compare Decomper with the state-of-the-art open-source structured pruning and low-rank decomposition methods, including LLM-Pruner (Ma et al., 2023), FLAP (An et al., 2024), SliceGPT (Ashkboos et al., 2024), AFM (Yu & Wu, 2023), ASVD (Yuan et al., 2023), Dobi-SVD (Wang et al., 2025b), SVD-LLM (Wang et al., 2025c) and Basis Sharing (Wang et al., 2025a). Methods like FLAP, LLM-Pruner, and ASVD lack support for OPT, and FLAP is incompatible with LLaMA-3’s GQA architecture. The implementation details are provided in Appendix C due to space limitations.

4.1. Overall Performance

Downstream tasks performance. Table 2, 11, and 12 show that Decomper delivers consistent and remarkable perfor-

Table 2. Performance comparison of different methods on LLaMA-2-7B/13B, Qwen3-8B, LLaMA-3-8B, and OPT-30B models. **Bold** and underline denote the best and second-best results, respectively. Extended results are provided in Appendix Table 11 and 12.

Methods	Ratio	PPL ($\downarrow$)	Avg. Acc. ($\uparrow$)	BoolQ	PIQA	WinoG.	HellaS.	ARC	OBQA	MMLU
LLaMA-2-7B	0%	5.469	64.10	77.77	79.05	69.38	75.92	60.37	44.2	45.7
LLM-Pruner	20%	12.375	<u>54.61</u>	63.58	76.22	62.59	68.55	<u>50.82</u>	41.0	23.3
SliceGPT		10.295	41.95	37.98	60.72	59.67	44.45	37.78	30.8	26.4
FLAP		<u>6.764</u>	53.18	53.94	74.54	62.98	64.74	48.86	<u>39.6</u>	<u>31.9</u>
ASVD		12.575	48.88	63.58	67.57	61.17	55.80	41.77	32.8	26.6
Dobi-SVD		9.389	46.04	55.11	66.27	58.48	51.55	38.41	33.0	27.1
SVD-LLM		8.074	47.28	54.86	66.00	62.04	53.53	39.09	36.6	27.0
Basis Sharing		7.704	51.72	57.49	71.22	<u>63.06</u>	59.90	48.33	39.4	26.0
Decomper		6.762	56.05	<u>62.57</u>	<u>74.97</u>	65.82	<u>65.88</u>	52.86	41.0	32.4
LLaMA-2-13B	0%	4.881	67.55	80.55	80.41	72.53	79.41	63.27	45.6	55.4
LLM-Pruner	20%	11.251	56.39	62.97	77.97	60.77	71.26	55.69	44.0	22.8
SliceGPT		8.696	44.88	37.86	62.24	63.54	47.30	39.25	38.6	31.0
FLAP		6.213	58.18	66.42	75.57	67.25	69.19	39.25	40.8	41.2
ASVD		7.945	55.49	<u>75.54</u>	73.34	66.46	63.27	46.44	39.8	32.6
Dobi-SVD		<u>5.832</u>	<u>61.84</u>	75.84	76.50	68.35	<u>71.37</u>	<u>55.69</u>	<u>43.4</u>	47.9
SVD-LLM		6.728	55.65	70.40	71.76	68.43	59.59	49.72	41.0	34.6
Basis Sharing		6.649	57.89	72.91	73.23	<u>69.61</u>	62.68	53.93	42.2	34.6
Decomper		5.468	61.90	71.71	<u>76.82</u>	69.06	72.66	59.34	42.2	<u>44.1</u>
LLaMA-3-8B	0%	6.140	69.35	80.95	80.85	73.01	79.13	65.47	45.0	64.9
LLM-Pruner	20%	14.662	50.39	60.70	72.47	63.69	<u>58.18</u>	43.00	34.4	27.7
SliceGPT		19.473	37.05	37.83	54.41	56.75	35.73	29.06	28.2	25.4
Dobi-SVD		14.930	48.01	63.39	64.42	63.06	49.39	39.75	34.4	29.9
SVD-LLM		17.233	50.63	68.41	66.27	65.98	50.81	44.09	33.6	31.8
Basis Sharing		<u>13.803</u>	<u>52.55</u>	67.66	69.42	66.30	56.48	46.88	39.2	27.6
Decomper		11.172	54.59	62.69	69.97	66.22	60.60	51.67	37.4	36.5
Qwen3-8B	0%	9.251	70.16	86.61	77.69	67.96	74.84	40.73	42.0	74.7
ASVD	20%	21.289	56.41	71.68	72.42	59.59	60.20	52.53	36.6	45.7
Dobi-SVD		<u>14.00</u>	55.17	74.37	68.88	63.69	56.03	46.05	36.0	<u>50.3</u>
SVD-LLM		15.119	58.13	75.14	70.35	65.04	55.86	<u>56.54</u>	35.4	50.2
Basis Sharing		14.908	<u>58.66</u>	<u>76.45</u>	<u>71.33</u>	<u>65.59</u>	61.99	53.53	38.8	48.1
Decomper		13.699	61.25	<u>77.65</u>	71.11	66.69	59.83	58.85	35.8	61.3
OPT-30B	0%	9.500	57.40	70.12	78.07	68.59	72.13	51.71	39.8	27.1
SliceGPT	30%	13.750	44.00	38.84	66.59	58.56	51.97	37.87	34.4	25.9
SVD-LLM		<u>12.188</u>	<u>53.70</u>	<u>61.10</u>	<u>76.06</u>	<u>63.69</u>	<u>68.60</u>	<u>47.86</u>	40.4	24.6
Decomper		10.125	55.74	68.50	76.82	66.46	69.43	50.29	<u>38.8</u>	<u>25.3</u>

mance recovery, achieved using only a general-domain Wiki corpus rather than task-related data. Our analysis reveals three key patterns: (1) *Superiority Over Strong Baselines*: Decomper outperforms not only state-of-the-art low-rank decomposition methods but also advanced structured pruning baselines. For instance, on LLaMA-2-7B at 20% compression, Decomper achieves an average accuracy of 56.05, establishing a significant margin over both the strongest decomposition baseline Basis Sharing (51.72) and the structured pruning method FLAP (53.18). (2) *General Effectiveness Across Tasks and Models*: Improvements are consistent across diverse reasoning tasks. On the challenging MMLU benchmark, Decomper boosts the accuracy of Qwen3-8B from 50.2 (SVD-LLM) to 61.3, effectively bridging the

gap. This trend of recovery is consistently observed across other model architectures. (3) *Robustness Under Higher Compression*: Decomper maintains much more graceful degradation under higher compression ratios. For example, on OPT-13B at 40% compression, Decomper sustains an accuracy of 52.67, significantly reducing the performance gap to the original model compared to SVD-LLM’s 42.60.

To verify stability, we evaluate variance over 3 random seeds for LLaMA-3-8B (20% ratio), yielding stable results: PPL 11.07 ± 0.11 and average common-sense accuracy 57.36 ± 0.20 . The negligible variance compared to the substantial performance margins over baselines confirms the robustness of our method.

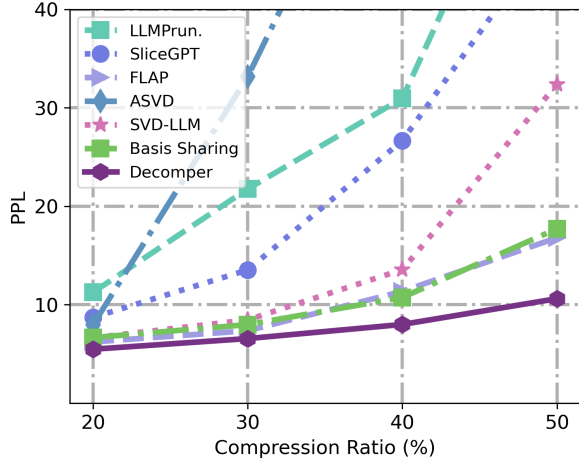


Figure 3. LLaMA-2-13B perplexity at 20–50% compression.

Generation quality. We report the perplexity scores to evaluate generation quality. Decomper consistently performs superior perplexity across various compression ratio settings on different model series. For instance, at 20% compression, Decomper reduces the perplexity of LLaMA-2-13B to 5.47, outperforming FLAP (6.21) and SVD-LLM (6.73). Similarly, on OPT-6.7B, Decomper dramatically lowers PPL from over 27 to 12.38. Figure 3 demonstrates that this performance improvement becomes more pronounced as the compression ratio increases, indicating a robust capability to maintain model generation quality. Extended results are shown in Appendix Table 11 and 12.

4.2. In-Depth Analysis

This section presents a comprehensive analysis of our compensation strategy. We first quantify the contribution of the compensation mechanism itself, then examine its robustness to calibration data variations. Furthermore, we demonstrate the practical utility of our approach through measurements of inference acceleration and compatibility with quantization techniques. Finally, we validate the generality of our method across different techniques and modalities.

Contribution of compensation strategy. As demonstrated in Table 3, our compensation strategy consistently improves both generation quality (measured by perplexity) and LMEH downstream task accuracy across various model architectures and compression ratios. Notable improvements include the LLaMA-3-8B model, which achieves a reduction in perplexity from 29.22 to 16.16 (a 45% relative improvement) at 30% compression ratio, and the OPT-13B model, which shows an accuracy increase from 48.6 to 56.6 (16% relative improvement). The benefits are even more substantial at higher compression ratios: for instance, OPT-6.7B at 40% compression ratio exhibits a dramatic perplexity reduction from 153 to 14.25 (91% improvement) while accuracy

Table 3. Comparison of perplexity and average accuracy of common-sense reasoning tasks without (w/o) and with (w/) our compensation strategy. The top and bottom blocks correspond to compression ratios of 30% and 40%, respectively.

Model	PPL ($\downarrow$)		Avg. Acc. ($\uparrow$)	
	w/o	w/	w/o	w/
LLaMA-2-7B	9.27	8.07 ($\downarrow 13\%$)	53.6	55.2 ($\uparrow 3\%$)
LLaMA-2-13B	7.52	6.55 ($\downarrow 13\%$)	56.9	59.6 ($\uparrow 5\%$)
LLaMA-3-8B	29.22	16.16 ($\downarrow 45\%$)	47.3	50.7 ($\uparrow 7\%$)
OPT-6.7B	41.25	12.38 ($\downarrow 70\%$)	47.1	55.1 ($\uparrow 17\%$)
OPT-13B	23.5	12.94 ($\downarrow 45\%$)	48.6	56.6 ($\uparrow 16\%$)
OPT-30B	12.19	10.13 ($\downarrow 17\%$)	57.9	60.1 ($\uparrow 4\%$)
LLaMA-2-7B	14.10	10.43 ($\downarrow 26\%$)	45.8	50.0 ($\uparrow 9\%$)
LLaMA-2-13B	9.80	8.09 ($\downarrow 17\%$)	51.7	55.8 ($\uparrow 8\%$)
LLaMA-3-8B	89.53	26.66 ($\downarrow 70\%$)	38.1	42.7 ($\uparrow 12\%$)
OPT-6.7B	153	14.25 ($\downarrow 91\%$)	39.4	52.0 ($\uparrow 32\%$)
OPT-13B	68.0	15.88 ($\downarrow 77\%$)	42.6	52.7 ($\uparrow 24\%$)
OPT-30B	13.8	10.75 ($\downarrow 22\%$)	55.4	58.6 ($\uparrow 6\%$)

Table 4. Perplexity of LLaMA-2-13B under 30% compression ratio using calibration data with different numbers and types.

Calibration Data	Number		
	512	1024	2048
WikiText2	6.95	6.91	6.87
C4	7.34	7.43	7.40

improves from 39.4 to 52.0 (32% relative improvement). These results confirm that our compensation strategy effectively mitigates performance degradation in decomposed models, with particularly significant gains observed under aggressive compression settings.

Robustness to calibration dataset. We analyze the effect of the calibration data, which corrects the latent representation distribution to preserve the model quality. Table 4 illustrates the perplexity score assessed on the WikiText2 test corpus, derived from variations in quantity and category of calibration data. The variations in language modelling performance due to the calibration dataset are minor, with perplexity from 6.87 to 7.43, indicating the number and type of calibration data have a limited impact on Decomper, and a small set of calibration data (e.g., 512 samples) can bridge the latent representation gap.

Inference speedup. To simulate a latency-sensitive deployment scenario, we measure the Time-To-First-Token (TTFT) latency in milliseconds. All evaluations are conducted on LLaMA-2 models using a single A800 (80 GB) GPU with a sequence length of 512. As shown in Figure 4, Decomper achieves notable speedups: a $1.25\times$ acceleration at 30% compression and $1.49\times$ at 50% compression for LLaMA-2-13B. Notably, this speedup is achieved entirely through the reduction of parameters and FLOPs resulting from decom-

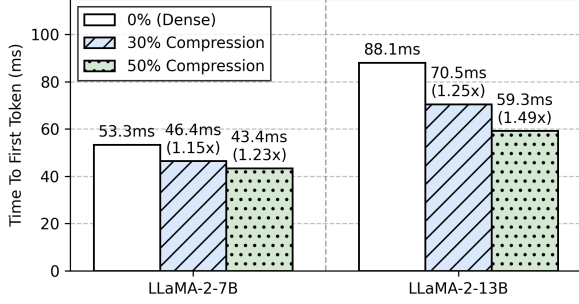


Figure 4. Inference speedup of time to first token.

Table 5. The synergy between decomposition and quantization is evident: Decomper (20%) + 4-bit achieves a lower perplexity than 3-bit quantization at a smaller memory footprint.

Method	Dense	GPTQ 4-bit	GPTQ 3-bit	Decomper + 4-bit
LLaMA-2-7B				
Memory (GB)	12.55	3.7	2.8	2.5
PPL ($\downarrow$)	5.47	6.22	6.96	6.55
LLaMA-3-8B				
Memory (GB)	14.96	5.4	4.6	4.5
PPL ($\downarrow$)	6.14	10.49	16.58	11.72

position. By fusing the c vectors into the bias terms, our method ensures no additional computational overhead.

Synergy with quantization. Quantization primarily targets numerical precision, yet realizing high-performance inference with low-bit kernels typically necessitates recent GPU architectures. In contrast, low-rank decomposition provides hardware-agnostic FLOPs reduction, which is orthogonal to the numerical compression of quantization. We demonstrate this synergy in Table 5. Combining 20% Decomper with 4-bit quantization achieves a superior trade-off compared to aggressive 3-bit quantization alone. For LLaMA-2-7B, it uses less memory (2.5 GB vs. 2.8 GB) while also achieving a better PPL (6.55 vs. 6.96). Similarly, for LLaMA-3-8B, the combined model is 41% better in PPL (11.72 vs 16.58) at comparable memory size (4.5 GB vs. 4.6 GB).

Generality to decomposition techniques. Our method is agnostic to the decomposition technique. Beyond SVD, we validate its effectiveness with PCA-based decomposition (AFM (Yu & Wu, 2023)). As detailed in Appendix D, our compensation yields consistent performance gains, e.g., recovering PPL from 18.82 to 10.44 on LLaMA-2-7B. This benefit translates to downstream tasks, boosting the average accuracy of LLaMA-2-13B by 9.4% at 30% compression.

Extension to vision language models. Beyond large language models and textual benchmarks, we validate our approach on vision-language tasks using Qwen2.5-VL-

Table 6. Comparison of different methods for LLaMA-2 series. “FT” denotes recovery fine-tuning, and “LS” denotes least-squares matrix updates.

Method	Ratio	PPL (↓)	Avg. Acc. (↑)
LLaMA-2-7B			
Dense	0%	5.47	64.1
FT (SliceGPT)	30%	9.39	41.4
FT (LLM-Pruner)		9.21	49.1
LS (SVD-LLM)		11.5	45.4
Our Compen.		8.07	51.7
FT (SliceGPT)	40%	13.6	37.9
FT (LLM-Pruner)		12.9	49.9
LS (SVD-LLM)		22.3	39.0
Our Compen.		10.4	50.0
LLaMA-2-13B			
Dense	0%	4.88	67.6
FT (SliceGPT)	30%	12.9	46.8
FT (LLM-Pruner)		9.01	51.9
LS (SVD-LLM)		8.45	51.2
Our Compen.		6.55	56.4
FT (SliceGPT)	40%	14.7	40.9
FT (LLM-Pruner)		13.3	46.9
LS (SVD-LLM)		13.2	45.2
Our Compen.		7.99	55.8

Instruct. Results in Appendix Table 8 show that our compensation improves COCO captioning quality across multiple metrics, with CIDEr increasing by 23.2% at 20% compression ratio and Bleu scores showing consistent gains, confirming the method’s versatility across modalities.

4.3. Comparison with Fine-tuning and Matrix Updates

We contrast our lightweight compensation with more parameter-intensive approaches: recovery fine-tuning (FT) and least-squares (LS) updates of decomposed matrices. We evaluate performance using Perplexity and Average Accuracy (averaged across the 8 reasoning and knowledge benchmarks detailed in Section 4.1). As shown in Table 6, our method, despite using far less compute for adaptation, outperforms these approaches, particularly at high compression ratios. This highlights the efficiency and superiority of our targeted compensation strategy over globally modifying weights. The fact that a simple, post-hoc bias correction can outperform these methods underscores the critical importance of directly addressing representation drift, rather than attempting to fine-tune away the task error.

5. Conclusion

In this work, we first diagnose a fundamental obstacle hindering the effectiveness of low-rank decomposition for

LLMs: the phenomenon of representation drift, where approximation errors in individual layers propagate and are non-linearly amplified through the transformer architecture. To mitigate this issue, we introduced a simple and effective compensation mechanism without additional inference overhead. Our method learns compensation terms that mitigate the initial injection of error and its subsequent propagation, effectively aligning the outputs of decomposed transformer blocks with their original counterparts. Extensive experiments demonstrate that our proposed compensation strategy consistently enhances the performance of decomposed models and exhibits strong compatibility and generality.

Acknowledgements

This work is supported by National Key R&D Program of China under Grant No. 2024YFA1012700. It is also funded by the NSFC Project (No. 62306256) and the Natural Science Foundation of Guangdong Province (No. 2025A1515010261).

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- An, Y., Zhao, X., Yu, T., Tang, M., and Wang, J. Fluctuation-based adaptive structured pruning for large language models. In *AAAI*, pp. 10865–10873. AAAI Press, 2024. URL <https://doi.org/10.1609/aaai.v38i10.28960>.
- Ashkboos, S., Croci, M. L., Nascimento, M. G. D., Hoeffler, T., and Hensman, J. SliceGPT: Compress large language models by deleting rows and columns. In *ICLR*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=vXxardq6db>.
- Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., et al. Qwen2.5-vl technical report. *CoRR*, arXiv:2502.13923, 2025. URL <https://arxiv.org/abs/2502.13923>.
- Chavan, A., Lele, N., and Gupta, D. K. Surgical feature-space decomposition of llms: Why, when and how? In *ACL*, pp. 2389–2400. Association for Computational Linguistics, 2024. URL <https://doi.org/10.18653/v1/2024.acl-long.130>.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. Qlora: Efficient finetuning of quantized llms. In *NeurIPS*, 2023.
- Dong, P., Li, L., Tang, Z., Liu, X., Pan, X., Wang, Q., and Chu, X. Pruner-Zero: Evolving symbolic pruning metric from scratch for large language models. In *ICML*. PMLR, 2024. URL <https://openreview.net/forum?id=1tRLxQzdep>.
- Frantar, E. and Alistarh, D. SparseGPT: Massive language models can be accurately pruned in one-shot. In *ICML*. PMLR, 2023. URL <https://proceedings.mlr.press/v202/frantar23a.html>.
- Frantar, E., Ashkboos, S., Hoeffler, T., and Alistarh, D. GPTQ: accurate post-training quantization for generative pre-trained transformers. In *ICLR*. OpenReview.net, 2023.
- Gao, L., Tow, J., Abbasi, B., Biderman, S., and Black. A framework for few-shot language model evaluation. *Version v0.0.1*. Sept, 10:8–9, 2021.
- Grattafiori, A., Dubey, A., Jauhri, A., and et al. The Llama 3 herd of models, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *CoRR*, abs/2009.03300, 2020. URL <https://arxiv.org/abs/2009.03300>.
- Hsu, Y., Hua, T., Chang, S., Lou, Q., Shen, Y., and Jin, H. Language model compression with weighted low-rank factorization. In *ICLR*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=uPv9Y3gmAI5>.
- Huang, X., Huang, Y., and Wen, Z. SoLA: Leveraging soft activation sparsity and low-rank decomposition for large language model compression. In *AAAI*, pp. 17494–17502. AAAI Press, 2025. URL <https://doi.org/10.1609/aaai.v39i16.33923>.
- Ji, Y., Xiang, Y., Li, J., Xia, Q., Ye, Z., Duan, X., Wang, Z., Chen, K., and Zhang, M. Adaptive feature-based low-rank compression of large language models via bayesian optimization. In *EMNLP (Findings)*, pp. 4152–4168. Association for Computational Linguistics, 2024. URL <https://aclanthology.org/2024.findings-emnlp.240>.
- Kaplan, J., McCandlish, S., Henighan, T., and et al. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Lin, C., Gao, S., Smith, J. S., Patel, A., Tuli, S., Shen, Y., Jin, H., and Hsu, Y. MoDeGPT: Modular decomposition for large language model compression. In *ICLR*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=8EfxjTCg2k>.

- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W., Wang, W., Xiao, G., Dang, X., Gan, C., and Han, S. AWQ: activation-aware weight quantization for on-device LLM compression and acceleration. In *MLSys*. mlsys.org, 2024.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. Microsoft coco: Common objects in context. In *European conference on computer vision*, pp. 740–755. Springer, 2014.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *ICLR*. OpenReview.net, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Ma, X., Fang, G., and Wang, X. LLM-Pruner: On the structural pruning of large language models. In *NeurIPS*, 2023.
- Makni, M., Behdin, K., Xu, Z., Ponomareva, N., and Mazumder, R. Hassle-free: A unified framework for sparse plus low-rank matrix decomposition for llms. *CoRR*, abs/2502.00899, 2025. URL <https://doi.org/10.48550/arXiv.2502.00899>.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. In *ICLR*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=Byj72udxe>.
- Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Jin, H., Chen, T., and Jia, Z. Towards efficient generative large language model serving: A survey from algorithms to systems. *ACM Computing Surveys*, 2023. URL <https://doi.org/10.48550/arXiv.2312.15234>.
- Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. A simple and effective pruning approach for large language models. In *ICLR*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=PxoFut3dWW>.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., and et al. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023. URL <https://doi.org/10.48550/arXiv.2307.09288>.
- van der Ouderaa, T. F. A., Nagel, M., van Baalen, M., Asano, Y. M., and Blankevoort, T. The LLM surgeon. In *ICLR*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=DYIIRgwg2i>.
- Wang, J., Chen, Y., Lin, I., Li, B., and Zhang, G. L. Basis sharing: Cross-layer parameter sharing for large language model compression. In *ICLR*. OpenReview.net, 2025a. URL <https://openreview.net/forum?id=gp32jvUquq>.
- Wang, Q., Ke, J., Tomizuka, M., Keutzer, K., and Xu, C. Dobi-svd: Differentiable SVD for LLM compression and some new perspectives. In *ICLR*. OpenReview.net, 2025b. URL <https://openreview.net/forum?id=kws76i5XB8>.
- Wang, X., Zheng, Y., Wan, Z., and Zhang, M. SVD-LLM: truncation-aware singular value decomposition for large language model compression. *ICLR*, OpenReview.net, 2025c. URL <https://doi.org/10.48550/arXiv.2403.07378>.
- Yang, A., Li, A., Yang, B., and et al. Qwen3 technical report. 2025. URL <https://arxiv.org/abs/2505.09388>.
- Yu, H. and Wu, J. Compressing transformers: Features are low-rank, but weights are not! In *AAAI*, pp. 11007–11015. AAAI Press, 2023. URL <https://doi.org/10.1609/aaai.v37i9.26304>.
- Yuan, Z., Shang, Y., Song, Y., Wu, Q., Yan, Y., and Sun, G. ASVD: activation-aware singular value decomposition for compressing large language models. *CoRR*, abs/2312.05821, 2023. URL <https://doi.org/10.48550/arXiv.2312.05821>.
- Zhang, S. and Pappayan, V. OATS: outlier-aware pruning through sparse and low rank decomposition. In *ICLR*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=DLDuVbxORA>.
- Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., and et al. OPT: open pre-trained transformer language models. *CoRR*, abs/2205.01068, 2022. URL <https://doi.org/10.48550/arXiv.2205.01068>.

A. Derivation of Reconstruction Loss

Here we provide the detailed derivation of the reconstruction loss presented in Equation 1. We begin by defining the loss function $\mathcal{L}(W_r)$ as the expected squared Euclidean norm of the error vector RX , where $R = W - W_r$ represents the residual matrix and X is the input activations. To tractably analyze this expectation, we utilize the properties of the trace operator and the statistical moments of X . The derivation proceeds as follows:

$$\begin{aligned}
\mathcal{L}(W_r) &= \mathbb{E}[\|RX\|_2^2] \\
&= \mathbb{E}[\text{Tr}((RX)(RX)^\top)] \quad (\text{since } \|v\|_2^2 = \text{Tr}(vv^\top)) \\
&= \mathbb{E}[\text{Tr}(RXX^\top R^\top)] \\
&= \text{Tr}(\mathbb{E}[RXX^\top R^\top]) \quad (\text{Linearity of Trace and Expectation}) \\
&= \text{Tr}(R \cdot \mathbb{E}[XX^\top] \cdot R^\top) \\
&= \text{Tr}(R(\Sigma + \mu\mu^\top)R^\top) \quad (\text{since } \mathbb{E}[XX^\top] = \Sigma + \mu\mu^\top) \\
&= \text{Tr}(R\Sigma R^\top + R\mu\mu^\top R^\top) \\
&= \text{Tr}(R\Sigma R^\top) + \text{Tr}(R\mu\mu^\top R^\top) \\
&= \text{Tr}(R\Sigma R^\top) + \text{Tr}((R\mu)(R\mu)^\top) \quad (\text{Trace property}) \\
&= \text{Tr}(R\Sigma R^\top) + \|R\mu\|_2^2
\end{aligned}$$

This derivation explicitly shows that the total error consists of a mean-shift component $\|R\mu\|_2^2$ and a variance-dependent component $\text{Tr}(R\Sigma R^\top)$.

B. Pseudocode of Decomper

Algorithm 1 details the execution of Decomper, structured as a sequential, block-by-block compression and alignment process. For each Transformer block l , we first apply truncated SVD to compress the weight matrices of all linear layers (e.g., query, key, value, and MLP projections), reducing W to W_r . This compression inevitably introduces approximation errors that distort feature distributions. To mitigate the resulting representation drift, we perform a post-hoc compensation optimization. Crucially, this step accounts for error propagation: the optimization for block l uses the output of the previous decomposed block (H_r^{l-1}) as input. We initialize learnable compensation vectors c for the layers in the current block and optimize them via gradient descent on a small calibration dataset D . The objective is to minimize the L_2 divergence between the accumulated decomposed output and the original target: $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \|H_r^l(H_r^{l-1}(x_i); c) - H^l(x_i)\|_2^2$. This formulation ensures that c not only corrects local errors but also aligns the drifted representations accumulated from prior blocks. Finally, the optimized c is fused into the bias term ($b' = b + c$) or added as a bias for bias-free layers, completing an enhancement for block l before moving to $l + 1$.

C. Experiment Implementation Details

All experiments are implemented using the HuggingFace Transformers library. In our main experiments, we employ data-whitening SVD (*i.e.*, SVD-LLM (Wang et al., 2025c)) as a representative instantiation. Specifically, we derive the whitening matrix L via the Cholesky decomposition of the calibration input covariance, defined as $\Sigma = \frac{1}{T} X^T X = LL^T$. By applying SVD to the whitened matrix WL instead of W , we explicitly account for input activation statistics. Our method is applicable to various LLM decomposition techniques; additional results with the PCA-based method AFM are included in the Appendix D due to space limitations.

The compensation terms are optimized using AdamW (Loshchilov & Hutter, 2019) with a cosine annealing scheduler and an initial learning rate of 0.005. We utilize a calibration set of 2,048 samples, each with a sequence length of 256. Following prior work (Hsu et al., 2022; Ma et al., 2023; Huang et al., 2025), compression ratios are allocated proportionally based on Fisher Information, excluding the initial and terminal layers. For the recovery fine-tuning and decomposed matrix updates discussed in Section 4.3, we adhere to established protocols, using 8,192 and 512 samples from the general-domain WikiText-2 dataset, respectively, both with a sequence length of 1,024. To ensure fair comparison, Dobi-SVD is evaluated

with quantization remapping disabled, and we report strict memory-based compression ratios. Finally, to conserve horizontal space, we condensed ARC-Easy and ARC-Challenge into a single column (“ARC”) displaying their mean. This accounts for the slight numerical deviation observed when averaging the visible columns.

Algorithm 1 Decomper: LLM Decomposition with Representation Drift Compensation

Input: Original LLM M_{orig} , Calibration dataset $D = \{x_k\}_{k=1}^N$

- 1: Initialize decomposed model M_{decomp} from M_{orig} using SVD truncation.
- 2: **for** each Transformer block l in the model **do**
- 3: // Get activations
- 4: Collect block inputs $X_{in} = \{H_r^{l-1}(x_k)\}$ from current M_{decomp} .
- 5: Collect target outputs $Y_{target} = \{H^l(x_k)\}$ from M_{orig} .
- 6: **for** each batch (x_{in}, y_{target}) in (X_{in}, Y_{target}) **do**
- 7: // Forward pass with frozen W_r and learnable c
- 8: Compute output: $y_{pred} = \text{Forward}(M_{decomp}.\text{block}_l, \text{input} = x_{in}, \text{params} = \{c\})$
- 9: Calculate loss: $\mathcal{L} = \|y_{pred} - y_{target}\|_2^2$
- 10: Update compensation: $c \leftarrow c - \eta \nabla_c \mathcal{L}$
- 11: **end for**
- 12: // Fuse
- 13: Fuse c into bias terms of block l in M_{decomp} .
- 14: **end for**

Output: Decomposed model M_{decomp}

D. Generalization to PCA-based Decomposition

To further validate the generality of our compensation framework, we integrate it with AFM, a PCA-based decomposition method (Yu & Wu, 2023). As summarized in Table 7, our compensation brings consistent and substantial improvements to AFM across models and compression ratios. On LLaMA-2-7B at 20% compression, perplexity drops from 8.41 to 6.70 and average accuracy rises from 55.38 to 59.86. More notably, under higher compression (40%), compensation recovers perplexity from 18.82 to 10.44 and accuracy from 42.05 to 50.16—narrowing the gap to the original model significantly. Similar trends hold for LLaMA-2-13B, where at 30% compression, accuracy improves from 49.53 to 58.93. These results confirm that our approach is decomposition-agnostic and effectively mitigates representation drift across diverse low-rank compression techniques.

Table 7. Performance of PCA-based decomposition (AFM) with and without compensation on LLaMA-2 models across compression ratios.

Methods	Ratio	PPL ($\downarrow$)	Avg. ($\uparrow$)	BoolQ	PIQA	WinoG.	HellaS.	ARC	OBQA
LLaMA-2-7B	0%	5.47	66.72	77.77	79.05	69.38	75.92	60.37	44.2
AFM	20%	8.41	55.38	65.75	68.72	64.88	54.34	47.68	38.6
w/ Compen.		6.70	59.86	62.57	74.81	65.67	66.94	53.82	41.4
AFM	30%	11.43	49.53	64.01	62.13	61.80	45.16	40.51	32.6
w/ Compen.		7.96	55.53	62.26	70.18	62.43	58.52	48.07	39.2
AFM	40%	18.82	42.05	49.69	56.58	57.30	36.38	32.81	28.8
w/ Compen.		10.44	50.16	61.1	65.13	59.43	49.0	41.13	34.2
LLaMA-2-13B	0%	4.88	69.30	80.55	80.41	72.53	79.41	63.27	45.6
AFM	20%	8.41	59.09	64.10	73.18	67.80	60.75	54.41	39.0
w/ Compen.		5.97	62.66	64.04	76.82	67.72	71.44	58.50	41.6
AFM	30%	11.43	49.53	64.01	62.13	61.80	45.16	40.51	32.6
w/ Compen.		7.08	58.93	62.29	73.01	66.30	64.57	53.97	38.4
AFM	40%	13.22	46.34	55.54	59.25	60.06	41.24	37.35	33.6
w/ Compen.		8.61	55.20	61.99	68.34	64.25	57.10	48.68	37.4

E. Vision Language Model Application

To evaluate the generalizability of our method beyond pure language modeling, we additionally evaluate on vision-language tasks. Specifically, we compress the Qwen2.5-VL-Instruct model and report its performance on the COCO dataset, a comprehensive benchmark for image understanding that involves object detection, segmentation, and captioning. This evaluation serves to validate whether our compensation mechanism effectively preserves capabilities across fundamentally distinct modalities and task structures.

The COCO evaluation results of the compressed Qwen2.5-VL-Instruct model (20% ratio), presented in Table 8, show that our compensation brings substantial gains in captioning quality, with CIDEr increasing from 0.2973 to 0.3662 (a 23% relative improvement) and Bleu-4 from 0.1126 to 0.1244. Bleu-n (Bilingual Evaluation Understudy) measures n-gram precision between generated and reference captions, with higher scores indicating greater similarity. CIDEr (Consensus-based Image Description Evaluation), specifically designed for image captioning, assesses the consensus between generated and reference descriptions using TF-IDF weighted n-grams; higher CIDEr scores reflect more relevant and informative captions. This consistent pattern of recovery—observed in common-sense reasoning, knowledge-intensive question answering, and complex vision-language tasks—collectively supports that our approach effectively mitigates representation drift, thereby preserving semantic fidelity in low-rank LLM decomposition.

Table 8. COCO evaluation results of Qwen2.5-VL-Instruct at 20% compression ratio.

Metric	w/o Compen.	w/ Compen.
Bleu_1	0.4342	0.4481 (↑3.2%)
Bleu_2	0.2802	0.2986 (↑6.6%)
Bleu_3	0.1776	0.1926 (↑8.4%)
Bleu_4	0.1126	0.1244 (↑10.5%)
CIDEr	0.2973	0.3662 (↑23.2%)

F. Ablation Study on Alignment Loss Functions

In our main methodology, we employ the L_2 norm as the optimization objective for the compensation mechanism. Here, we provide a justification for this choice by comparing it with other potential alignment strategies:

- **KL/JS Divergence:** These probabilistic metrics are generally unsuitable for aligning hidden states in Transformer blocks. Hidden representations often contain negative values (e.g., outputs from GeLU/SiLU activations or linear projections), which are outside the domain of logarithmic operations required by KL/JS divergence, leading to numerical instability (NaN values).
- **Wasserstein Distance:** While theoretically sound for measuring distribution shifts, estimating the Wasserstein distance is computationally expensive and difficult for efficient optimization. In contrast, L_2 loss offers the ideal balance.
- **Cosine Similarity:** We empirically compared our L_2 loss with Cosine Similarity loss ($\mathcal{L} = 1 - \cos(H, H_r)$). As shown in Table 9, while Cosine Similarity is a reasonable alternative, the L_2 loss consistently yields better perplexity and downstream accuracy.

Table 9. Comparison of different loss functions for the compensation mechanism on LLaMA-2-7B.

Model	Loss Function	20% Compression		30% Compression	
		PPL (↓)	Avg. Acc. (↑)	PPL (↓)	Avg. Acc. (↑)
LLaMA-2-7B	Cosine Similarity	6.94	59.09	8.32	55.09
	L_2 Loss	6.76	59.43	8.07	55.16

G. Optimization Efficiency and Convergence

While the optimization problem in Equation 6 is non-convex due to the Transformer architecture, we empirically observe stable and rapid convergence. For LLaMA-3-8B optimization, the block-level L_2 loss typically drops steeply within the first 4 batches (e.g., from 12.40 to 7.36) and quickly converges to a stable minimum (approximately 2.99, a 76% reduction). Figure 5 visualizes this stable trajectory across different architectures.

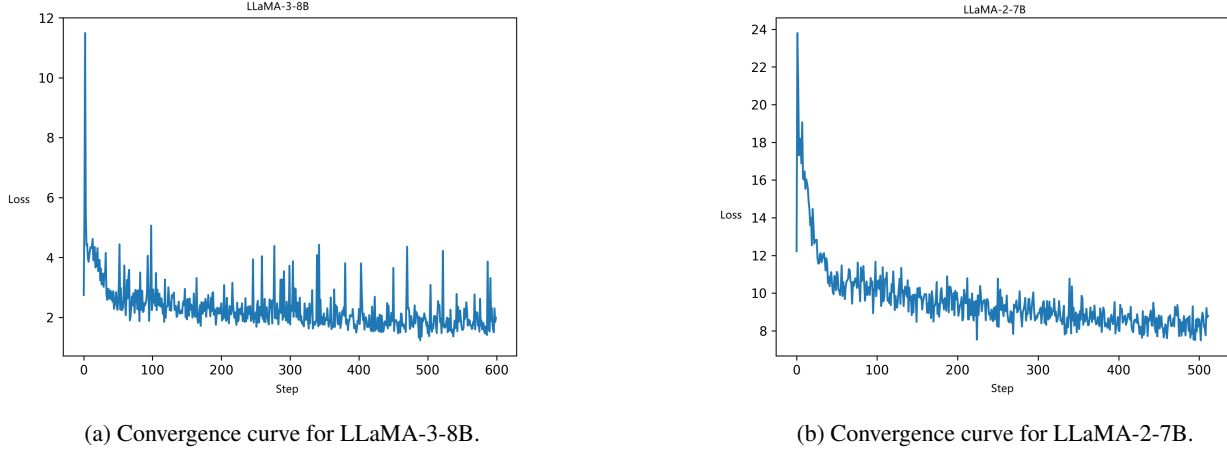


Figure 5. Empirical convergence of our optimization method across different model architectures.

Computational Cost. We further summarize the wall-clock time of the compensation optimization phase and memory overhead of the compensation vectors in Table 10.

- **Time Cost:** The optimization process is highly efficient as it is performed offline. Since we only optimize low-dimensional vectors c (e.g., $\mathbb{R}^{4096}$) while keeping the large model weights frozen, the computational load is minimal. For instance, the process completes for LLaMA-2-13B in approximately 2 hours, with the time cost scaling roughly linearly with model size.
- **Memory Cost:** For architectures already equipped with bias terms (e.g., OPT series), our method incurs **zero** additional memory overhead as the compensation is absorbed into existing parameters. For bias-free architectures (e.g., LLaMA), the learned compensation vectors consume negligible memory. For instance, in a 13B model (24.24 GB in Bfloat16), the additional parameters amount to only 6.5 MB—less than 0.03% of the total model size.

Table 10. Efficiency analysis showing minimal time and negligible memory overhead.

Model	Wall-clock Time	Compensation Vectors Memory
OPT-6.7B	~ 50 mins	0 (Fused into existing bias)
OPT-13B	~ 80 mins	0 (Fused into existing bias)
OPT-30B	~ 3 h	0 (Fused into existing bias)
LLaMA-2-7B	~ 75 mins	4.4 MB
LLaMA-2-13B	~ 2 h	6.5 MB

H. Extended Results on Higher Compression Ratios

Table 11 and Table 12 provide comprehensive results across multiple compression ratios (20%, 30%, and 40%) for various model families, including LLaMA-3-8B, LLaMA-2-7B, LLaMA-2-13B, Qwen3-8B, OPT-6.7B, and OPT-13B. Methods such as LLM-Pruner, FLAP, ASVD, and Dobi-SVD are not applicable to the OPT architecture. These extended results demonstrate the robustness of Decomper under different compression settings. In particular, Decomper consistently achieves

the best or competitive performance in both perplexity (PPL) and downstream task accuracy, even at higher compression ratios (e.g., 40%). For instance, on OPT-6.7B at 40% compression, Decomper maintains strong performance with a PPL of 14.25 and an average accuracy of 51.97. This significantly surpasses the performance of SVD-LLM, which yields a PPL of 153 and an accuracy of only 39.40 under the same conditions. Similarly, on LLaMA-2-7B at 40% compression, Decomper attains a perplexity of 10.43 and an average accuracy of 50.00, significantly outperforming methods like ASVD (PPL NaN, Acc 37.94) and SVD-LLM (PPL 14.10, Acc 45.87). These results further validate the effectiveness and stability of the proposed compensation mechanism in preserving model capabilities under different levels of compression.

I. Additional Latent Representation Distributions

In this appendix, we extend our investigation of representation drift to the LLaMA-2-7B and LLaMA-2-13B models to further illustrate the effects of low-rank decomposition and the benefits of our compensation mechanism. To highlight the effect of error propagation and amplification—which becomes more severe in deeper layers due to the cumulative effect of approximation errors through the network—we deliberately analyze Transformer blocks located in the middle-to-late stages of each model. Specifically, we visualize the absolute values of the latent representations (showing the 50th percentile, 95th percentile, and maximum value per channel) for: LLaMA-3-8B: The 23rd Transformer block. LLaMA-2-7B: The 25th Transformer block (out of 32 total blocks). LLaMA-2-13B: The 33rd Transformer block (out of 40 total blocks).

These figures mirror the analysis presented in Figure 6 for LLaMA-3-8B and demonstrate similar trends across different model architectures. The distribution of latent representations for the original models, the low-rank decomposed models, and the models enhanced with our compensation mechanism are shown. The figures display the 50th percentile, 95th percentile, and maximum value for each channel across the Transformer blocks. Our compensation mechanism effectively counteracts this drift, restoring the distribution of latent values closer to that of the original model. This recovery is evident across all percentiles, supporting our claim that the proposed method mitigates representation degradation and contributes to improved model performance. The consistent pattern of representation drift and its mitigation through our compensation strategy across various models further validates the general applicability of our approach.

J. Limitations and Future Work

While the proposed method demonstrates superior suitability for LLM deployment in resource-constrained environments, we explicitly distinguish its intended application scenarios based on the trade-off between structural speedup and reasoning precision. *Recommended Scenarios:* Decomper is highly effective for latency-sensitive applications requiring general language understanding, knowledge retrieval, and open-ended generation. In these areas, it provides essential FLOPs reduction with minimal quality loss. *Cautionary Scenarios:* For brittle, high-precision tasks like complex mathematical reasoning, capabilities tend to degrade rapidly under any aggressive compression—whether via decomposition or low-bit quantization. Consequently, we advise that decomposition in these domains should be applied with caution or paired with subsequent recovery instruction tuning.

The concept of representation drift and its mitigation via output alignment, while explored here in the context of low-rank decomposition, likely has broader implications for model compression in general. We believe it is a universal phenomenon in model compression. Our work offers a potential starting point for further research into “compress-then-align” strategies. Future work could explore automated compensation strategies for various compression techniques, including low-rank decomposition and pruning, or investigate extending the alignment objective to the distributional level. We hope this work inspires further investigation into these promising directions.

Table 11. WikiText2 perplexity and common-sense reasoning task accuracy of the LLaMA-2/3 models at different compression ratios.

Methods	Ratio	PPL ($\downarrow$)	Avg. ($\uparrow$)	BoolQ	PIQA	WinoG.	HellaS.	ARC	OBQA
LLaMA-3-8B	0%	6.14	69.98	80.95	80.85	73.01	79.13	65.47	45.0
LLM-Pruner	30%	<u>22.47</u>	46.73	53.30	67.90	57.85	<u>47.92</u>	33.77	<u>32.6</u>
SliceGPT		33.67	35.70	37.83	51.90	51.14	30.46	25.99	26.6
SVD-LLM		29.22	<u>47.33</u>	<u>61.28</u>	61.43	59.19	41.71	<u>39.35</u>	29.0
Basis Sharing		26.00	45.38	49.76	61.37	<u>60.14</u>	41.48	36.86	31.2
Decomper		17.26	50.65	63.49	<u>65.07</u>	61.33	48.63	41.60	32.8
LLM-Pruner	40%	<u>51.58</u>	<u>42.15</u>	52.39	62.08	<u>55.09</u>	<u>38.67</u>	28.10	30.6
SliceGPT		60.20	35.05	37.83	51.85	50.51	28.50	25.02	26.6
SVD-LLM		89.53	38.09	41.07	55.93	53.75	30.88	<u>29.51</u>	26.0
Basis Sharing		75.65	37.14	37.86	52.77	54.46	30.99	27.96	28.0
Decomper		26.66	42.65	<u>47.13</u>	<u>59.68</u>	58.41	38.72	33.02	<u>28.6</u>
LLaMA-2-7B	0%	5.47	66.72	77.77	79.05	69.38	75.92	60.37	44.2
LLM-Pruner	30%	18.88	50.96	<u>52.84</u>	72.00	54.62	<u>56.94</u>	41.44	37.0
SliceGPT		16.51	39.03	37.83	55.60	54.54	35.06	31.67	28.4
FLAP		<u>8.91</u>	<u>52.11</u>	52.20	<u>70.29</u>	60.06	56.58	<u>43.72</u>	<u>38.2</u>
ASVD		33.15	46.41	50.70	52.56	52.01	28.76	26.24	22.8
Dobi-SVD		12.38	44.21	44.22	66.27	58.88	42.57	33.27	31.0
SVD-LLM		11.52	45.37	49.88	61.04	58.64	43.48	34.98	34.6
Basis Sharing		9.69	48.64	43.76	66.49	<u>61.64</u>	50.38	41.19	35.8
Decomper		8.07	55.16	62.35	<u>70.29</u>	62.98	58.47	46.82	38.4
LLM-Pruner	40%	61.16	44.90	<u>59.85</u>	64.04	53.51	40.29	33.09	31.4
SliceGPT		27.41	36.31	37.83	52.23	51.46	30.92	27.99	25.8
FLAP		14.27	44.77	41.71	66.10	54.22	47.50	34.24	35.4
ASVD		Nan	37.94	37.98	49.40	51.30	26.27	26.03	24.4
SVD-LLM		14.10	<u>45.87</u>	51.87	62.19	59.04	42.86	<u>36.16</u>	32.8
Basis Sharing		<u>13.87</u>	43.89	40.09	61.10	59.83	41.62	35.19	34.2
Decomper		10.43	50.00	61.99	<u>65.13</u>	<u>59.12</u>	<u>49.25</u>	39.95	<u>34.6</u>
LLaMA-2-13B	0%	4.88	69.30	80.55	80.41	72.53	79.41	63.27	45.6
LLM-Pruner	30%	21.75	50.20	62.11	<u>73.18</u>	57.93	60.89	44.38	41.4
SliceGPT		13.52	41.27	37.83	56.75	57.70	38.27	33.53	31.6
FLAP		<u>7.35</u>	<u>57.30</u>	<u>64.37</u>	72.42	63.93	<u>62.44</u>	<u>49.37</u>	39.2
ASVD		33.15	46.41	66.30	64.15	57.85	44.21	36.21	35.6
SVD-LLM		8.45	48.35	<u>64.37</u>	65.61	62.98	47.81	39.91	37.6
Basis Sharing		7.97	55.38	66.42	67.57	<u>65.27</u>	52.57	48.21	<u>39.4</u>
Decomper		6.56	59.57	63.55	73.72	66.06	65.71	54.89	38.2
LLM-Pruner	40%	30.95	39.91	39.66	61.92	52.17	38.66	27.89	31.2
SliceGPT		26.66	36.94	37.83	52.77	52.88	30.74	28.38	27.6
FLAP		11.35	<u>54.14</u>	63.39	69.21	60.93	<u>55.99</u>	<u>45.12</u>	39.2
ASVD		462.4	37.94	57.71	54.08	49.64	28.26	26.24	23.4
SVD-LLM		13.55	51.71	61.74	64.96	<u>62.12</u>	50.07	43.05	37.0
Basis Sharing		<u>10.73</u>	49.14	60.80	62.79	61.96	42.82	39.82	36.0
Decomper		7.99	55.83	<u>62.51</u>	<u>68.34</u>	62.43	58.23	50.57	<u>38.2</u>

Table 12. WikiText2 perplexity and common-sense reasoning task accuracy of the decomposed models (Qwen and OPT series) at different compression ratios.

Methods	Ratio	PPL ($\downarrow$)	Avg. ($\uparrow$)	BoolQ	PIQA	WinoG.	HellaS.	ARC	OBQA
Qwen3-8B	0%	9.25	69.51	86.61	77.69	67.96	74.84	40.73	42.0
SVD-LLM	30%	20.03	46.45	63.12	62.30	54.62	41.46	36.94	29.8
Basis Sharing		<u>18.42</u>	<u>55.35</u>	77.80	<u>66.49</u>	<u>61.80</u>	53.21	<u>46.57</u>	35.0
Decomper		14.85	55.66	<u>75.05</u>	67.14	61.96	<u>52.98</u>	49.16	<u>34.2</u>
SVD-LLM	40%	42.82	38.25	37.86	56.75	52.33	34.13	29.66	27.4
Basis Sharing		<u>27.54</u>	<u>46.20</u>	<u>54.25</u>	<u>60.72</u>	59.35	<u>42.13</u>	<u>37.96</u>	31.0
Decomper		19.06	48.51	62.91	61.37	<u>58.88</u>	45.06	40.78	<u>29.8</u>
OPT-6.7B	0%	10.94	58.04	70.12	78.07	68.59	72.13	51.71	39.8
SliceGPT	20%	17.77	43.41	37.86	61.59	57.22	41.63	35.99	33.6
SVD-LLM		16.38	<u>51.55</u>	<u>57.52</u>	<u>71.16</u>	59.43	54.26	<u>41.14</u>	<u>36.2</u>
Decomper		11.44	55.97	60.83	75.41	62.90	64.79	45.84	36.2
SliceGPT	30%	28.83	38.65	37.83	55.01	53.43	33.25	31.51	28.0
SVD-LLM		27.88	<u>47.13</u>	<u>57.19</u>	<u>67.57</u>	55.56	41.35	<u>38.12</u>	<u>32.0</u>
Decomper		12.38	55.06	65.35	73.23	62.19	60.82	44.42	35.0
SliceGPT	40%	65.38	35.07	37.83	50.82	49.25	28.85	26.06	26.6
SVD-LLM		153	<u>39.40</u>	<u>40.18</u>	<u>58.54</u>	50.51	31.58	<u>32.78</u>	<u>29.4</u>
Decomper		14.25	51.97	60.86	70.78	59.98	55.88	40.64	35.0
OPT-13B	0%	10.13	59.12	65.29	76.82	64.88	69.79	48.94	39.2
SlicGPT	20%	13.78	46.50	39.54	66.59	58.80	51.03	37.97	33.6
SVD-LLM		12.75	<u>54.58</u>	<u>52.17</u>	<u>75.19</u>	<u>62.19</u>	<u>65.68</u>	<u>45.03</u>	<u>36.8</u>
Decomper		10.25	56.94	59.20	75.79	64.09	68.69	47.12	36.6
SlicGPT	30%	18.90	41.93	37.95	61.37	53.99	40.20	34.61	30.8
SVD-LLM		20.75	<u>48.58</u>	<u>55.17</u>	<u>65.67</u>	53.75	<u>51.89</u>	<u>39.98</u>	<u>33.6</u>
Decomper		12.94	56.62	64.74	73.39	62.43	65.18	45.52	39.6
SlicGPT	40%	39.09	37.14	37.83	54.03	51.85	29.95	28.96	28.4
SVD-LLM		68.00	<u>42.60</u>	<u>54.04</u>	<u>61.26</u>	<u>49.49</u>	<u>38.77</u>	<u>33.03</u>	<u>28.6</u>
Decomper		15.88	52.67	58.69	70.73	59.04	59.98	42.64	35.0

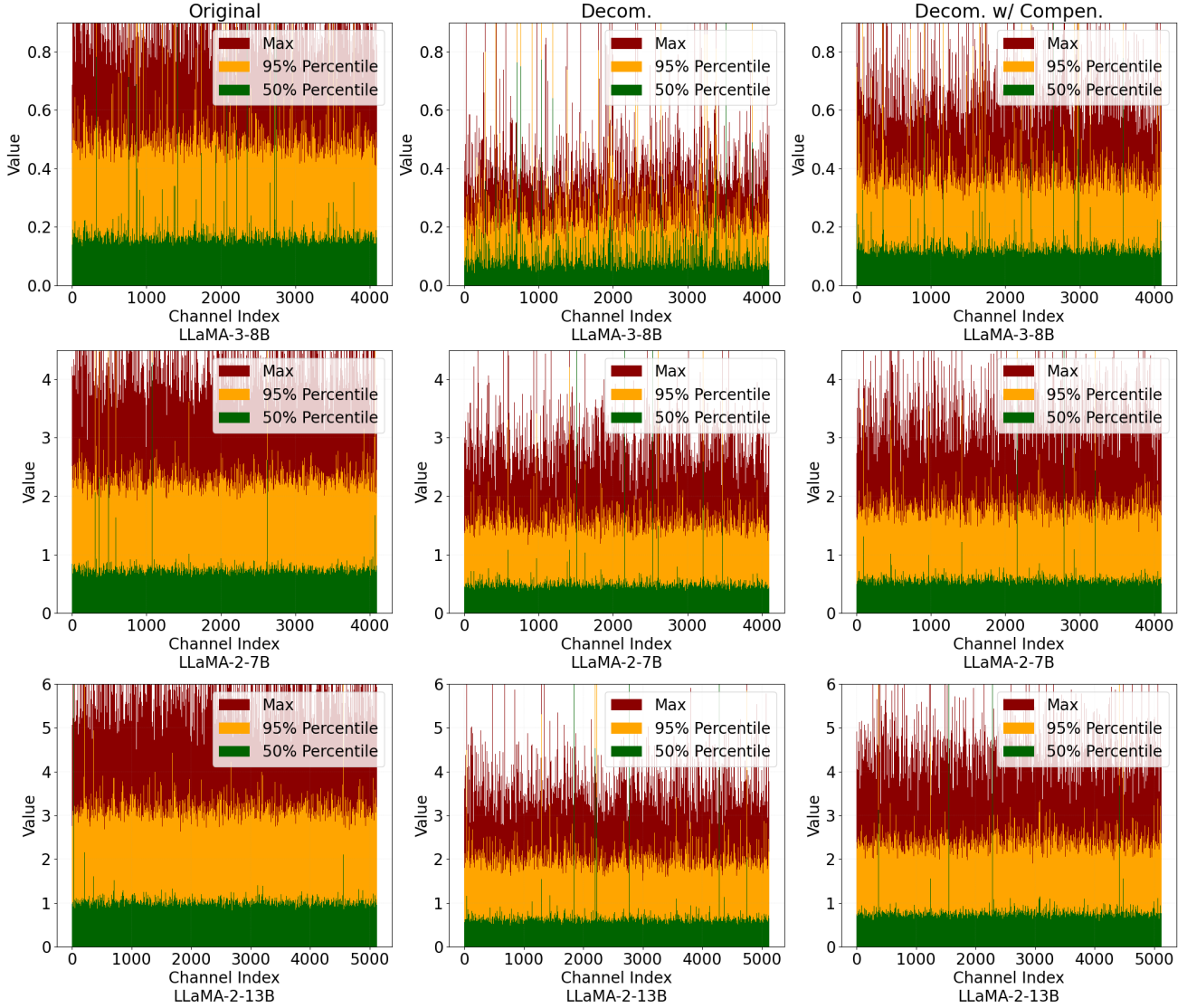


Figure 6. Latent representation drift in LLaMA-3-8B, LLaMA-2-7B, and LLaMA-2-13B decomposition and our compensation. Columns 1, 2, and 3 show the latent representations (Transformer Block outputs) of the original model, the decomposed model, and the decomposed model with compensation, respectively.